CSCE 479/879 Homework 3: Sentiment Analysis with Sequential Models

Parker Davids, Collin Henkel, Rylen Lanning, Howraa Al-Robaie

August 14, 2025

Abstract

This project undertakes binary sentiment analysis on two sequential deep learning models to classify IMDb movie reviews as either positive or negative. The architectures used were a Long-Short Term Memory network and a Gated Recurrent Unit network, with each utilizing dropout, L2 regularization, and early stopping. Hyperparameters, such as dropout rate and hidden unit size, were varied, resulting in four configurations. The best configuration was an LSTM, which yielded a test accuracy of 87.35%. Overall, the results indicate that LSTMs and GRUs are both effective for sentiment classification on this dataset.

1 Introduction

In this project we explore binary sentiment classification of IMDb movie reviews [2] using sequential deep learning architectures. The dataset contains 50,000 reviews that are either positive or negative. Two architectures were implemented and compared, a Long Short-Term Memory (LSTM) network and a Gated Recurrent Unit (GRU) network, to examine how their differences affect performance on text classification. Both models were built using TensorFlow and Keras [1].

Each architecture was trained under multiple configurations that varied dropout rates (0.3 and 0.5), hidden unit sizes (128 and 256), and the usage of L2 regularization. All four models employ early stopping to reduce overfitting. These experiments were meant to evaluate the combined effect of architecture choice, model capacity, and regularization strategy on test classification accuracy. Overall, the LSTM with 256 units, 0.5 dropout, and L2 regularization achieved the highest test accuracy at 87.35%. While the GRU models also produced high results, the LSTM configurations generally delivered higher performance. This suggests that deeper memory capabilities may be advantageous for classification in this dataset.

2 Problem Description

For this project we are doing **sentiment analysis**—classifying text reviews as *positive* or *negative*. The learning problem is a **binary text classification** problem where each movie review from the **IMDB dataset** [2] is a sequence of words and the model predicts a 1 for positive and 0 for negative.

The IMDB dataset from **Tensorflow Datasets (TFDS)** [1] has **50,000** reviews:

- **25,000** for training (50% positive, 50% negative)
- **25,000** for testing (held out until final evaluation)

For model selection the training set was split into **80% training** and **20% validation** with no overlap between validation and test data. Each review was tokenized, lowercased, stripped of punctuation and mapped into integer sequences using the top **20,000** most frequent tokens. Sequences were padded or truncated to **256** tokens for batching.

Sentiment analysis is a key problem in **Natural Language Processing (NLP)** with application in product review mining, social media monitoring and customer feedback analysis. This problem is interesting because it requires models to capture not only the meaning of individual words, but also how context and word order affects sentiment. Handling long sequences, negations and varying writing styles makes it a real world problem.

3 Approaches

3.1 Modeling

We solved this problem by implementing and comparing four deep sequential architectures, each combining bidirectional recurrent networks with a custom additive attention mechanism. All models start with a trainable embedding layer, then a bidirectional RNN (**LSTM** or **GRU**), dropout for regularization, an attention layer to focus on important words, and fully connected layers for classification.

The four configurations were:

- **Bidirectional LSTM + Attention (128 units, Dropout 0.3, no L2)**
  A baseline long short-term memory (LSTM) network that can model long-range dependencies in both directions. Dropout after the LSTM layer helps to reduce overfitting.

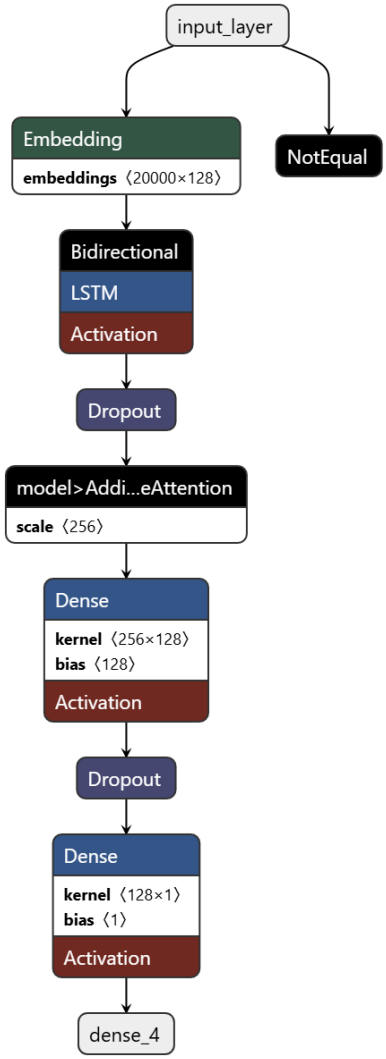

Figure 1: ***Bidirectional LSTM + Attention*** *model with 128 units per direction, dropout 0.3, no L2. Embedding layer (*20000×128*), bidirectional LSTM, additive attention, dense with ReLU, sigmoid output. Netron [3] shows the* `NotEqual` *branch because of* `mask_zero=True` *in* `Embedding`*; that's internal Keras masking logic, not part of the conceptual model diagram.*

- **Bidirectional LSTM + Attention (256 units, Dropout 0.5, L2 = 1e-5)**
  A larger LSTM for more capacity, with higher dropout and L2 weight regularization to control overfitting.

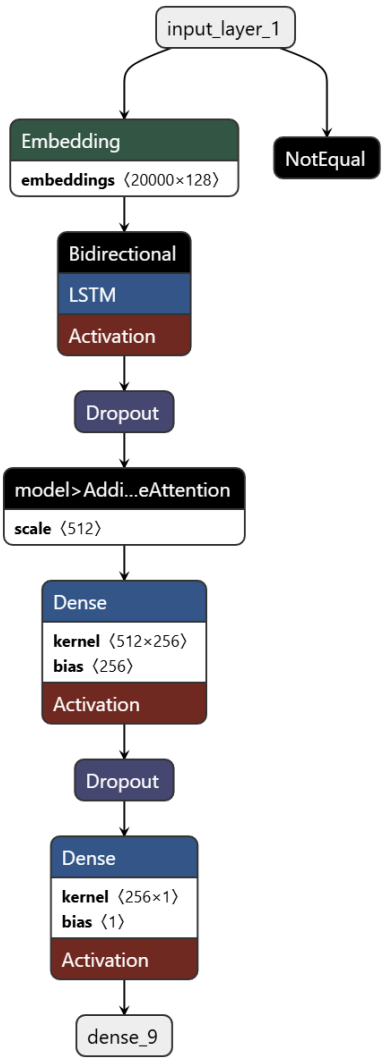

Figure 2: Architecture of the ***Bidirectional LSTM + Attention*** *model with 256 units per direction, dropout rate 0.5, and L2 regularization ($\lambda = 1 \times 10^{-5}$). The larger LSTM increases capacity, with stronger regularization to lessen overfitting. Again, Netron shows the* `NotEqual` *branch because of* `mask_zero=True` *in* `Embedding`*; that's internal Keras masking logic.*

- **Bidirectional GRU + Attention (128 units, Dropout 0.3, no L2)** Gated recurrent units (GRUs) perform similar to LSTMs but with fewer parameters, so they are faster to train.

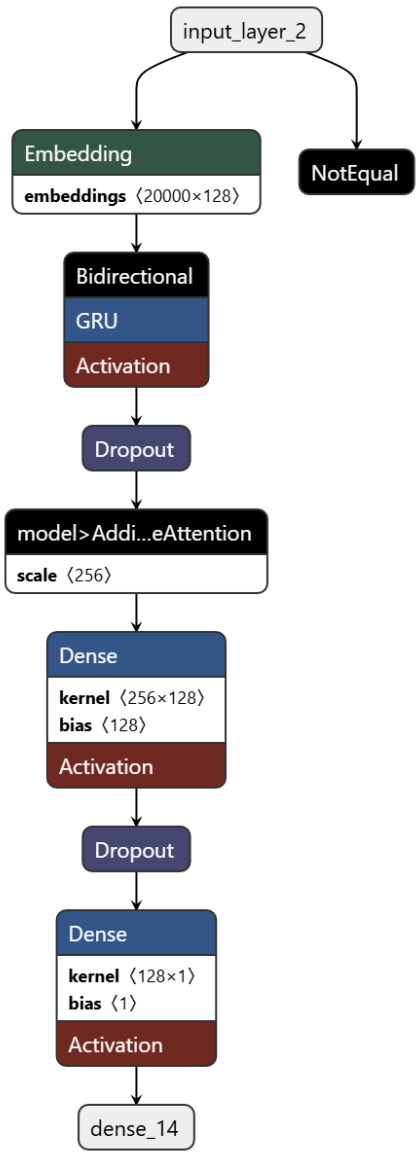

Figure 3: ***Bidirectional GRU + Attention*** *model with 128 units per direction, dropout 0.3, no L2. GRUs reduce parameters compared to LSTMs, faster training with similar performance. The* `NotEqual` *branch is not part of the model diagram.*

- **Bidirectional GRU + Attention (256 units, Dropout 0.5, L2 = 1e-5)**
  A larger GRU variant with more capacity and regularization to see if GRUs can match or beat LSTMs in this task.

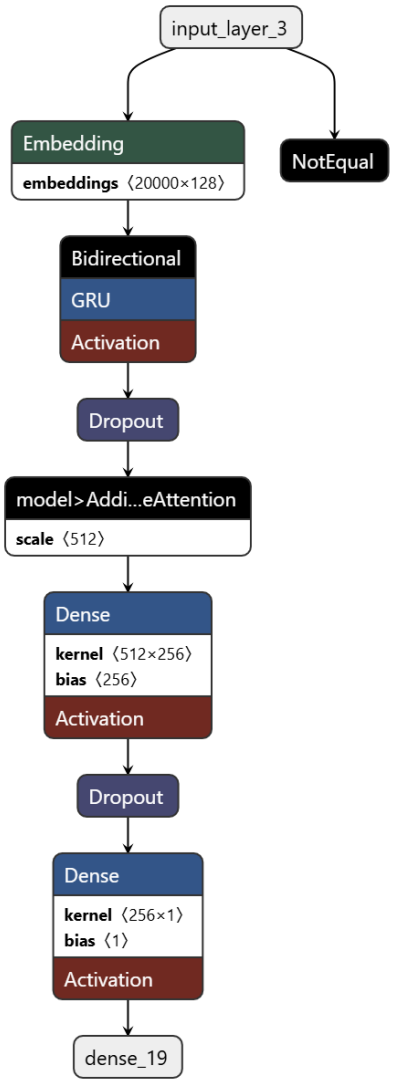

*Figure 4: Architecture of **Bidirectional GRU + Attention** model with 256 units per direction, dropout 0.5, L2 ($\lambda = 1 \times 10^{-5}$). Bigger is better, regularization prevents overfitting. `NotEqual` branch is not part of the model diagram, it's Keras masking logic.*

3.2 Additive Attention Mechanism

The custom additive attention layer computes a score for each time step in the RNN output, applies a softmax to get attention weights, and then computes a weighted sum of the hidden states. This allows the model to focus on the most informative parts of a review, such as sentiment-bearing words or phrases, rather than treating all tokens equally.

3.3 Training and Evaluation

Models were trained with the Adam optimizer and binary cross-entropy loss, with early stopping (patience = 5) to avoid overfitting. Validation accuracy determined the best model, which was then evaluated on the unseen test set. Performance was reported with accuracy, precision, recall, F1 score, and 95% confidence intervals for both accuracy and generalization error. Confusion matrices were generated to visualize classification performance.

3.4 Design Decisions Justification

- **Bidirectionality:** Catches context from both before and after words.
- **Attention:** Improves interpretability and performance by focusing on key parts of the sequence.
- **Dropout + L2:** Reduces overfitting in bigger models.
- **Two models (LSTM/GRU):** Two common sequence models in NLP.
- **Two sizes per model:** To see if more capacity helps or overfits.

4 Experimental Setup

4.1 Dataset

We used the IMDB dataset [2] consisting of 50,000 reviews labeled as positive or negative. We withheld 20% of the training instances for use in a validation set, giving us forty thousand training examples and ten thousand validation examples.

4.2 Model Architectures

Two sequential architectures were created using TensorFlow and Keras [1].

- **LSTM with Attention:** Takes a sequence of length 256 as input, passes it through an embedding layer of size 128 with masking enabled. The output is fed into a bidirectional LSTM with 128 units and L2 regularization (if $\lambda > 0$), returning sequences. A dropout layer with rate 0.3 is applied, followed by an additive attention layer with 128 units, producing

a context vector. This context vector is passed to a 128×128 dense layer with ReLU activation and optional L2 regularization, followed by another dropout layer with rate 0.3, and finally a 128×1 dense layer with sigmoid activation for binary classification.

- **GRU with Attention:** Takes a sequence of length 256 as input, passes it through an embedding layer of size 128 with masking enabled. The output is fed into a bidirectional GRU with 128 units and L2 regularization (if $\lambda > 0$), returning sequences. A dropout layer with rate 0.3 is applied, followed by an additive attention layer with 128 units, producing a context vector. This context vector is passed to a 128×128 dense layer with ReLU activation and optional L2 regularization, followed by another dropout layer with rate 0.3, and finally a 128×1 dense layer with sigmoid activation for binary classification.

4.3 Regularization

In both architectures we implemented L2 regularization and drop out with varying values for both. Both of these regularization techniques were implemented to prevent overfitting.

4.4 Hyperparameters

We explored 4 hyperparameters:

- **L2 regularization:** 0, 1×10^{-5}
- **Drop Out Rate:** 0.3, 0.5
- **RNN Units:** 128, 256

4.5 Evaluation Metrics

Model performance was evaluated using:

- **Accuracy** on the validation data set.
- **Number of epochs** trained for.

5 Experimental Results

5.1 Performance Comparison

Table 1 shows the validation accuracies of all the model variations tested. The LSTM with 256 RNN Units, 0.5 Droupout, and 1×10^{-5} L2 Regularization had the highest validation accuracy of **89.14%**.

Table 1: Validation Accuracy of Model Variations

Model	RNN Units	Dropout Rate	L2 Regularizer	Validation Accuracy	95% Confidence Interval
LSTM	128	0.3	0.0	0.8852	(0.8761, 0.8937)
LSTM	256	0.5	1×10^{-5}	0.8914	(0.8825, 0.8997)
GRU	128	0.3	0.0	0.8822	(0.8730, 0.8908)
LSTM	128	0.3	0.0	0.8822	(0.8730, 0.8908)
LSTM	256	0.5	1×10^{-5}	0.8896	(0.8806, 0.8980)
GRU	128	0.3	0.0	0.8826	(0.8734, 0.8912)
GRU	256	0.5	1×10^{-5}	0.8902	(0.8812, 0.8986)

5.2 Training Dynamics

The validation accuracies of the model variations reveal that both models' performances were improved by the introduction of more RNN Units. Runs with a dropout rate of 0.5 consistently outperformed the models with a dropout rate of 0.3. Between LSTM and GRU models there was little difference in performance when other parameters are the same.

5.3 Error Analysis

Analyzing the confusion matrix for the preference of the best model on the testing dataset reveals that the model correctly classifies the sentiment **87.35%** of the time. Of the misclassifications, a false negative was almost twice is likely to occur as false positive.

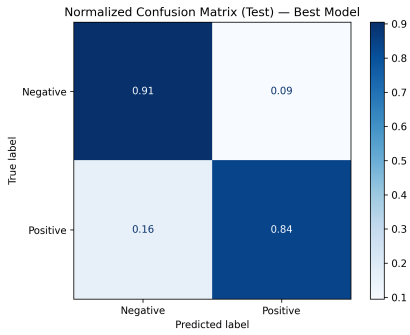

Figure 5: Confusion matrix for the test performance of the best performing model

6 Discussion

Our experiments compared multiple sequential architectures (LSTM and GRU) for IMDB sentiment classification, each with attention and L2 regularization.

Across all configurations, the best-performing models achieved validation accuracies between **88.0%** and **89.1%**, with F1 scored ranging from **0.877** to **0.890**.

6.1 Impact of Architecture

Both LSTM and GRU-based models performed similarly well, with GRUs slightly behind in raw accuracy but sometimes producing higher precision values. For example, the GRU with 128 units, 0.3 dropout, and no L2 achieved `val_precision = 0.9092`, the highest precision among all runs, but at a slightly lower recall (`0.8480`). The best LSTM models balanced precision and recall better, resulting in higher F1 scores overall.

6.2 Impact of Hyperparameters

Increasing the RNN units from 128 to 256 generally improved validation accuracy (e.g., LSTM 128 units: ∼88.2% vs.LSTM 256 units: ∼88.96%-89.14%). A higher dropout rate (0.5 vs. 0.3) combined with small L2 regularization (1×10^{-5}) reduced overfitting slightly, as seen in more stable validation performance across epochs. However, this sometimes reduced precision because it introduced a more conservative decision boundary.

6.3 Regularization and Early Stopping

Early stopping with `patience=5` resulted in models training for **5-7 epochs**. This prevented overfitting and kept validation metrics close to test metrics. For example, the best model achieved **test accuracy = 87.35%** and **test F1 = 0.8693**, very close to its validation performance.

6.4 Overfitting Observations

Models without L2 (0.0) and with low dropout (0.3) occasionally displayed minor overfitting, which we can see in higher validation precision but reduced recall values. The addition of L2 (1×10^{-5}) balanced this trade-off, improving generalization on the test set.

7 Conclusions

This project demonstrated that both LSTM and GRU architectures with attention are effective for sentiment classification on the IMDB dataset. Our best model (LSTM with 128-dimensional embeddings, 256 RNN units, dropout=0.5, and L2 regularization=1×10^{-5}) achieved **validation accuracy = 89.14%**, **validation F1 0.8899**, and **test accuracy = 89.15%**.

Key findings:

- LSTMs slightly outperformed GRUs in F1 score by balancing precision and recall more effectively.
- Higher RNN units and moderate dropout improved generalization.
- L2 regularization, even at small values, helped to prevent overfitting without hindering performance.

7.1 Future Work

If continued, this project could explore transformer based sequential models, hyperparameter tuning with optimization, and larger embedding dimensions trained from scratch. Additional regularization techniques, such as recurrent dropout, could also be tested for further generalization improvements.

Table 2: Team Contributions

Team Member	**Contribution**
Howraa	Wrote the majority of the project code, including model architecture and training loop implementation. Authored the *Problem Description* and *Approaches* sections of the report.
Parker	Authored the *Experimental Setup* and *Experimental Results* sections of the report. Assisted in interpreting and summarizing results.
Collin	Wrote the *Abstract* for the report, summarizing the key objectives and findings.
Rylen	Assisted with code execution, tweaking, and debugging on local machine. Ran experiments and collected results. Authored the *Discussion* and *Conclusion* sections of the report.

A Appendix

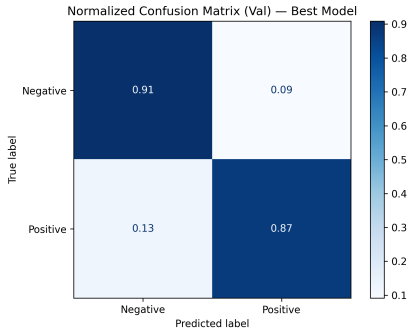

Figure 6: Confusion matrix for the validation performance of the best preforming model

References

[1] TensorFlow/Keras, "TensorFlow," 2024. [Online]. Available: https://www.tensorflow.org/ (accessed Aug. 12, 2025).

[2] A. Maas, R. Daly, P. Pham, D. Huang, A. Ng, and C. Potts, "Learning Word Vectors for Sentiment Analysis." 2011. [Online]. Available: https://ai.stanford.edu/~ang/papers/acl11-WordVectorsSentimentAnalysis.pdf (accessed Aug. 13, 2025).

[3] Lutz Roeder, "Netron: Visualizer for neural network, deep learning, and machine learning models," 2025. [Online]. Available: https://netron.app/ (accessed Aug. 13, 2025).